Customer Requirement Discovery Call Transcript

Project: BOM Validation and Anomaly Detection

Call Type: Initial requirement discussion
Customer: Manufacturing and Product Engineering Organization
Participants:

- **Amjad** – Solution Architect
- **Sarah Khan** – Customer PLM Manager
- **David Lee** – Customer Manufacturing Systems Lead
- **Rohit Mehta** – Customer IT Integration Lead
- **Maria Gomez** – Customer Product Data Governance Lead

Transcript

Amjad:
Thanks everyone for joining. The purpose of today's call is to understand the current challenges around your BOM data and what kind of solution you are expecting. We'll keep this as a discovery discussion first, and then we can translate it into requirements later.

Sarah:
Sure. From our side, the main issue is that our engineering teams maintain product structures in Oracle Fusion PLM, but before those structures are used downstream, we often find data quality issues. Some are simple, like missing UOMs or invalid quantities. Others are harder to detect, especially when the BOM has multiple levels.

Amjad:
When you say product structures, are you referring mainly to manufacturing BOMs, engineering BOMs, or both?

Sarah:
Mostly engineering BOMs at this point, but eventually the same concept should help manufacturing as well. Right now, our immediate concern is before release. We want to catch issues before a product structure is approved or sent further downstream.

David:
Exactly. A bad BOM creates problems later in planning, costing, procurement, and sometimes even on the shop floor. By the time manufacturing notices it, the issue has already moved too far.

Amjad:
Understood. What are some examples of the issues you are seeing?

Maria:
There are a few common ones. Sometimes the same component appears more than once under the same parent item. Sometimes quantity is zero or blank. In some cases, UOM is missing. We also see obsolete components still being used in active or released structures.

David:
We've also seen strange quantities. For example, a screw that is normally used in quantities like 4, 6, or 8 suddenly appears as 800. It may be a typo, but no one notices until later.

Amjad:
So you want both rule-based validations and some kind of anomaly detection?

Sarah:
Yes, but we want to keep it practical. We are not asking for a complex data science model right now. Even basic anomaly detection would help if it highlights unusual BOMs or unusual component quantities.

Rohit:
From IT side, we would prefer the solution to work with Oracle Integration Cloud. We already use OIC for several integrations, so if data needs to be pulled from Fusion PLM, OIC should be involved.

Amjad:
Would the solution need to update Fusion PLM as well, or is it only for reporting and review?

Sarah:
For phase one, reporting and review is fine. We don't want the system automatically changing PLM data. At most, maybe it can generate a recommendation or comment, but a human should review it.

Maria:
Yes, we are sensitive about automatic updates. AI should not directly update product structures. It can help explain issues, but it should not become the decision-maker.

Amjad:
That makes sense. What kind of user interface are you expecting?

Sarah:
We would like a dashboard where users can see which BOMs have issues. Something like a health score would be useful, but we are open to suggestions. The main thing is that engineers should be able to quickly find risky BOMs.

David:
A BOM detail view would also be useful. If I click on a parent item, I should see the components and the issues associated with them. It does not need to be very fancy, but it should be usable.

Amjad:
Would a tree-style BOM view be required?

David:
It would be nice. But if that takes too much effort, even a parent-child table is acceptable for phase one. The important part is that multi-level structures should not be ignored.

Rohit:
We also want the application to be built using Oracle Visual Builder, since our team is trying to standardize lightweight internal apps on VBCS.

Amjad:
Understood. So Visual Builder for the front end. What about the database layer?

Rohit:
Use Oracle Autonomous Transaction Processing. We can use ATP as the staging and reporting database. We don't want the dashboard to directly call Fusion PLM every time because performance and API limits may become a problem.

Amjad:
How should Visual Builder access the ATP data?

Rohit:
ORDS would be preferred. We already expose some ATP data using ORDS REST APIs, so that pattern is acceptable.

Amjad:
Got it. Let's talk about AI. You mentioned AI should explain issues, not directly change data. Can you elaborate?

Maria:
A lot of validation messages are too technical. For example, if a rule says "component has invalid effectivity," the engineer still has to figure out the impact. We want AI to explain the issue in simple business language.

Sarah:
Something like: "This component is obsolete and should not be used in a released BOM because it may affect procurement or manufacturing readiness." That kind of explanation would be helpful.

David:
Also, if there are many issues in one BOM, AI could summarize the overall risk. For example, "This BOM is not ready for release because it has two critical issues and three warnings."

Amjad:
Would you expect AI to recommend corrective actions?

Maria:
Yes, but only as suggestions. For example, if UOM is missing, it can suggest checking the primary UOM from the item master. If a component is obsolete, it can recommend replacing it with an active revision or approved substitute, but it should not decide the substitute by itself.

Amjad:
That is clear. Are there any specific validations that must be part of the first version?

Sarah:
We definitely need duplicate components, missing UOM, invalid quantity, obsolete components in active BOMs, and effectivity date issues.

David:
Please also consider circular references or BOM loops. We know Oracle normally tries to prevent that, but during migrations or external loads, we have seen bad structures in staging before they reached the final system. It would be good to detect that early.

Amjad:
So the system should be able to detect a parent item appearing again somewhere in its own lower-level structure?

David:
Exactly. If ITEM-A contains ITEM-B, and ITEM-B contains ITEM-C, and ITEM-C somehow contains ITEM-A, that should be flagged as critical.

Rohit:
Even if Fusion PLM prevents some of these cases, the staging layer should still validate them. We may load files or external extracts in the future, not only live PLM data.

Amjad:
That’s helpful context. How much data should we assume for the first version?

Rohit:
For a prototype, a few hundred BOMs is fine. We don’t need production scale right now. But the design should not be completely hardcoded to 10 records only.

Sarah:
Agreed. We can provide sample data if required, but the solution should have a way to load or sync data.

Amjad:
Would the source always be Fusion PLM, or should the application support file-based/mock data too?

Rohit:
Fusion PLM is the target source. However, for development, mock data or sample payloads are acceptable if access is limited. But we still want the integration pattern to be realistic.

Amjad:
So ideally OIC should either call Fusion PLM or process a mock payload that resembles Fusion PLM data.

Rohit:
Yes, that would be acceptable.

Amjad:
How should validation be triggered? Scheduled, manual, or both?

Sarah:
Both would be useful, but if we have to prioritize, scheduled validation is important. Maybe once a day or on demand for a selected BOM.

David:
Manual validation is useful when an engineer wants to check one item before release.

Amjad:
What should happen after validation runs?

Maria:
The system should store the results. We want to see history, not just the latest status. If a BOM had issues last week and now it is clean, that history is useful.

Rohit:
Yes, please log validation runs. Also integration logs are important. If OIC fails to pull data from PLM, we need to know why.

Amjad:
What statuses do you expect for issues?

Sarah:
Maybe Open, Reviewed, Ignored, and Resolved. We are not strict about the naming, but users should be able to track whether someone looked at the issue.

Maria:
If someone marks an issue as ignored, they should provide a comment. Some duplicate components may be intentional, although usually they are not.

Amjad:
That’s a good point. Should all duplicate components be treated as errors?

David:
Not always. It depends. Sometimes there may be legitimate reasons, such as different operation sequences. But we still want them flagged for review.

Sarah:
Severity should probably depend on the issue type. Missing UOM or circular reference is critical. Duplicate component may be warning or high depending on the case.

Amjad:
Do you already have severity rules defined?

Sarah:
Not fully. We have some ideas, but we expect the implementation team to propose a reasonable severity model.

Amjad:
Understood. Do you want a score, like BOM health score?

Sarah:
Yes, that would be helpful, but we do not have a formula. The team can propose something simple and explainable.

Maria:
Please avoid a black-box score. If the health score is 40, users should know why.

Amjad:
Good point. Explainability matters.

Rohit:
Also, please make sure the system separates actual validation rules from AI-generated recommendations. We don't want someone thinking an AI suggestion is the same as an approved business rule.

Amjad:
That is important. AI should support the decision, not replace the rule engine.

David:
Correct. Also, for anomaly detection, keep it understandable. If a quantity is unusually high, explain what it was compared against. Similar item class, historical average, or configured threshold — something like that.

Amjad:
Do you have historical BOM data available?

Sarah:
Some, but not perfectly clean. For the prototype, assume we can provide enough sample data to compare against. We mainly want to see the concept working.

Amjad:
What reports or dashboards would leadership want to see?

Sarah:
Top risky BOMs, issue count by severity, issue count by rule, and maybe BOM health by item class.

Maria:
Also, show which issues are still open. We don't want a dashboard that only shows counts but does not help users take action.

Amjad:
Do you need notifications?

Sarah:
Not for phase one. Nice to have later.

Rohit:
Agreed. No email or workflow needed initially unless the team has extra time.

Amjad:
What about security? Any role-based access?

Rohit:
For now, basic role separation is enough. Admin users can run validation and manage rules. Business users can view and review issues. But this does not need to be enterprise-grade security for the first version.

Amjad:
Should validation rules be configurable from the UI?

Sarah:
That would be nice, but I don't want it to delay the main functionality. At minimum, rules should not be completely buried in a way that no one can understand them.

Maria:
Even a table that stores rule names, descriptions, and severity would be useful.

Amjad:
Understood. Are there any expectations around documentation?

Rohit:
Yes. We need a short design document, API list, table list, integration flow, and known limitations. Nothing too massive, but enough for our internal team to understand.

Amjad:
For the demo, what would you like to see?

Sarah:
Start from the dashboard. Show that some BOMs are healthy and some are risky. Then open one risky BOM, show the components and the validation issues. Then show AI explaining one or two issues.

David:
Please include a multi-level BOM example. If the system only works for one-level BOMs, it will not be very useful.

Maria:
And include at least one case where the system detects something unusual, not just missing fields.

Amjad:
For example, a quantity anomaly?

Maria:
Yes. Or a BOM that has far more components than expected for that type of item.

Rohit:
From IT side, I also want to see that OIC processing is logged. If something fails, the support person should have enough information to troubleshoot.

Amjad:
Understood. Any constraints around timelines?

Sarah:
We are expecting a prototype in about three weeks. It does not need to be production-ready, but it should be end-to-end enough to demonstrate value.

Amjad:
When you say end-to-end, do you mean from PLM extraction all the way to Visual Builder dashboard?

Sarah:
Yes, ideally. But if live PLM access is blocked, then realistic mock payloads are acceptable. The architecture should still show where PLM fits.

Rohit:
Correct. We don’t want a standalone UI with manually entered rows only. Integration must be part of the solution.

Amjad:
That’s clear. Any final expectations?

Sarah:
Keep it practical. We want something our engineers can understand and use. Don’t overcomplicate the AI part. Focus on identifying bad BOM data and helping users understand what to fix.

Maria:
And please make sure recommendations are explainable. Users will not trust a system that just says “high risk” without telling them why.

David:
Also, do not assume every flagged issue is automatically wrong. Some cases require review. The application should support that.

Rohit:
From my side: use Oracle Integration Cloud, ATP, ORDS, and Visual Builder. AI should be included, but with proper boundaries. Also include logs and error handling.

Amjad:
Great. I'll summarize what I heard: you need a prototype solution that can bring BOM structure data into ATP, validate it for data quality and structural issues, identify anomalies, expose the results through APIs, show them in a Visual Builder dashboard, and use AI to explain issues and suggest actions. The solution should be practical, explainable, and safe, with no automatic PLM updates in phase one.

Sarah:
Yes, that captures it well.

Amjad:
Perfect. We'll take this input and convert it into a proposed solution approach and project scope.

Additional Notes Mentioned During the Call

- Customer prefers Oracle-native components where possible.
- Oracle Fusion PLM is the intended source system.
- Mock data is acceptable if live PLM access is delayed.
- OIC should be used for integration orchestration.
- ATP should store staged BOM data, validation results, and logs.
- ORDS should expose REST APIs for the application.
- Visual Builder should be used for the user interface.
- AI should explain, summarize, and recommend — not directly update PLM.
- Multi-level BOM handling is expected.
- Rule-based validation is more important than complex machine learning.
- The prototype should be achievable within approximately three weeks.